

```
!pip install tensorflow matplotlib scikit-image
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.12/dist-packages (0.25.2)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.0)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.80.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (1.16.3)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (3.6.1)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (2.37.3)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (2026.3.3)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (0.5)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.42.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.19.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.11.11)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.7.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (0.7.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.1.0)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (2.19.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files Brain MRI Dataset.zip
Brain MRI Dataset.zip(application/x-zip-compressed) - 15828590 bytes, last modified: 4/5/2026 - 100% done
 Saving Brain MRI Dataset.zip to Brain MRI Dataset.zip

```
import zipfile, os

zip_path = list(uploaded.keys())[0]

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('/content/mri_data')

print("Extracted!")
```

Extracted!

```
import cv2

def load_images(path, size=(64,64)):
    images = []

    for root, dirs, files in os.walk(path):
        for file in files:
            if file.endswith(('png', '.jpg', '.jpeg')):
                img_path = os.path.join(root, file)

                img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
                img = cv2.resize(img, size)

                img = img / 255.0
                images.append(img)

    images = np.array(images)
    images = images[..., None] # Add channel dimension

    return images

x_data = load_images('/content/mri_data')
print("Total images:", x_data.shape)
```

Total images: (358, 64, 64, 1)

```
split = int(0.8 * len(x_data))

x_train = x_data[:split]
x_test = x_data[split:]
```

```
def add_noise(images):
    noise = np.random.normal(0, 0.05, images.shape).astype(np.float32)
    noisy = images + noise
    return np.clip(noisy, 0., 1.).astype(np.float32)

x_train_noisy = add_noise(x_train)
x_test_noisy = add_noise(x_test)
```

```
def build_generator():
    inputs = layers.Input(shape=(64,64,1))

    x = layers.Conv2D(64, 3, padding='same', activation='relu')(inputs)
    x = layers.Conv2D(64, 3, padding='same', activation='relu')(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(128, 3, padding='same', activation='relu')(x)

    x = layers.UpSampling2D()(x)
    x = layers.Conv2D(64, 3, padding='same', activation='relu')(x)

    outputs = layers.Conv2D(1, 3, padding='same', activation='sigmoid')(x)

    return tf.keras.Model(inputs, outputs)
generator=build_generator()
```

```
def build_discriminator():
    model = models.Sequential([
        layers.Conv2D(32, 3, strides=2, padding='same', input_shape=(64,64,1)),
        layers.LeakyReLU(),

        layers.Conv2D(64, 3, strides=2, padding='same'),
        layers.LeakyReLU(),

        layers.Flatten(),
        layers.Dense(1, activation='sigmoid')
    ])
    return model

discriminator = build_discriminator()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape` to `super().__init__(activity_regularizer=activity_regularizer, **kwargs)`
```

```
cross_entropy = tf.keras.losses.BinaryCrossentropy()

def d_loss(real, fake):
    real_loss = cross_entropy(tf.ones_like(real), real)
    fake_loss = cross_entropy(tf.zeros_like(fake), fake)
    return real_loss + fake_loss
def g_loss(fake, generated, real):
    gan_loss = cross_entropy(tf.ones_like(fake), fake)

    l1_loss = tf.reduce_mean(tf.abs(real - generated))

    ssim_loss = 1 - tf.reduce_mean(tf.image.ssim(real, generated, max_val=1.0))

    return gan_loss + 150*l1_loss + 50*ssim_loss

generator_optimizer = tf.keras.optimizers.Adam(0.0002)
discriminator_optimizer = tf.keras.optimizers.Adam(0.0002)
```

```
@tf.function
def train_step(noisy, real):

    noisy = tf.cast(noisy, tf.float32)
    real = tf.cast(real, tf.float32)

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated = generator(noisy, training=True)

        real_output = discriminator(real, training=True)
        fake_output = discriminator(generated, training=True)

        gen_loss = g_loss(fake_output, generated, real)
        disc_loss = d_loss(real_output, fake_output)

        gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
        gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

        generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
        discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))

    return gen_loss, disc_loss
```

```
EPOCHS = 10
BATCH_SIZE = 16

train_dataset = tf.data.Dataset.from_tensor_slices((x_train_noisy, x_train))
train_dataset = train_dataset.shuffle(1000).batch(BATCH_SIZE)

for epoch in range(EPOCHS):
    ...print(f"\nEpoch {epoch+1}")

    ...for noisy_batch, real_batch in train_dataset:
    ..... g_loss_val, d_loss_val = train_step(noisy_batch, real_batch)

    ...print(f"G Loss: {g_loss_val.numpy():.4f}, D Loss: {d_loss_val.numpy():.4f}")
```

```
Epoch 1
G Loss: 82.7887, D Loss: 0.9957

Epoch 2
G Loss: 53.0389, D Loss: 0.7173

Epoch 3
G Loss: 34.4770, D Loss: 1.2193

Epoch 4
G Loss: 27.0904, D Loss: 1.2475

Epoch 5
G Loss: 23.5628, D Loss: 1.2531

Epoch 6
```

G Loss: 22.5437, D Loss: 1.2502

Epoch 7

G Loss: 21.2408, D Loss: 1.1573

Epoch 8

G Loss: 21.7729, D Loss: 1.1743

Epoch 9

G Loss: 21.4538, D Loss: 1.2107

Epoch 10

G Loss: 20.0017, D Loss: 1.1654

```
test_img = x_test_noisy[0:1]
```

```
denoised = generator.predict(test_img)
```

```
plt.figure(figsize=(12,4))
```

```
plt.subplot(1,3,1)
```

```
plt.title("Noisy MRI")
```

```
plt.imshow(test_img[0].squeeze(), cmap='gray')
```

```
plt.subplot(1,3,2)
```

```
plt.title("Denoised MRI")
```

```
plt.imshow(denoised[0].squeeze(), cmap='gray')
```

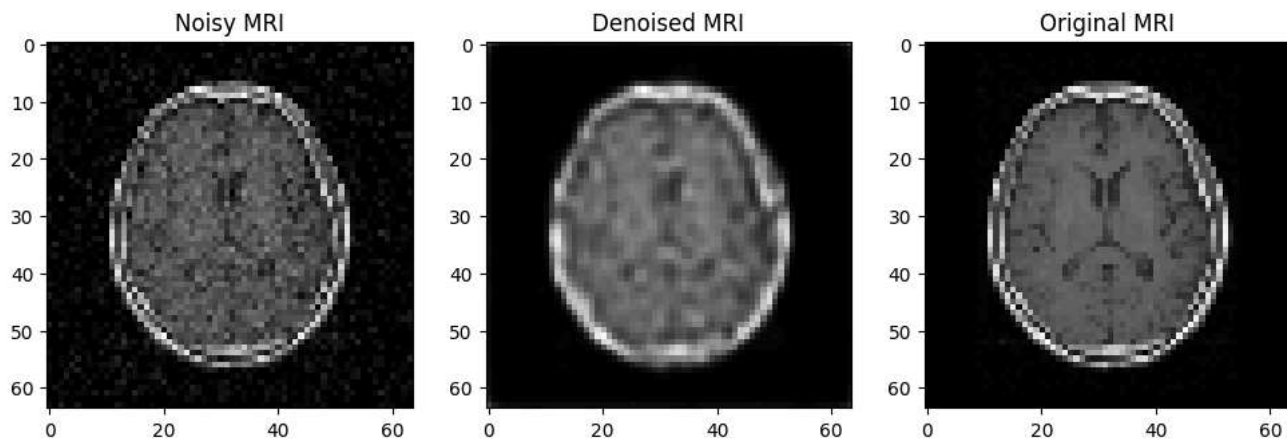
```
plt.subplot(1,3,3)
```

```
plt.title("Original MRI")
```

```
plt.imshow(x_test[0].squeeze(), cmap='gray')
```

```
plt.show()
```

1/1 ————— 1s 768ms/step



```
psnr_val = psnr(x_test[0], denoised[0], data_range=1)
```

```
ssim_val = ssim(x_test[0].squeeze(), denoised[0].squeeze(), data_range=1)
```

```
print("PSNR:", psnr_val)
```

```
print("SSIM:", ssim_val)
```

PSNR: 24.355444743435278

SSIM: 0.7687875780326943

